Design of Hardware Accelerators

Lab Lesson 2

Elia Lazzeri
Dipartimento di Elettronica, Informazione e Bioingegneria
Politecnico di Milano

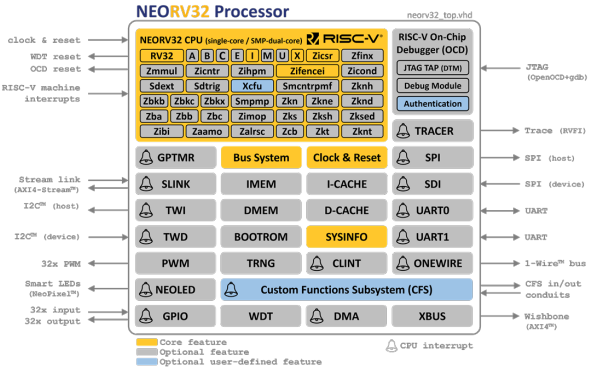
April 12, 2026

Today's Laboratory Session

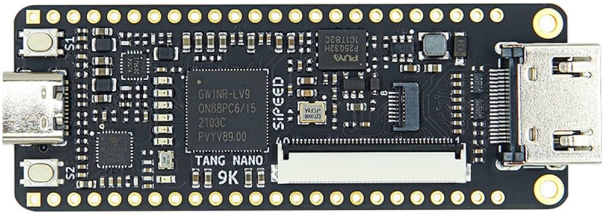
- Environment Setup: Familiarize with the Dockerized toolchain and VSCode workspace.
- Architecture Exploration: Understand the internal structure and configuration parameters (generics) of the NEORV32 core.
- Digital Design Workflow: Execute the full path from RTL simulation to FPGA bitstream generation.
- Hands-on Task: Modify the System-on-Chip (SoC) hardware parameters and measure the impact on area and performance.

The NEORV32 RISC-V Processor

- Open-Source & VHDL-Native
- Highly Modular: Features are enabled/disabled via VHDL generics without changing the source code.
- Standard Interfaces: Utilizes the Wishbone bus for standard memory and peripheral interconnects.
- Includes built-in UART, Timers, GPIOs, and a Custom Functions Subsystem (CFS) for future accelerators.



Tang Nano 9k FPGA



- Gowin GW1NR-9 FPGA
- Available Resources (The Bottleneck):
  - 8,640 LUTs (Look-Up Tables)
  - 6,480 Flip-Flops
  - 26 Block RAMs (BRAM)
- A fully loaded NEORV32 can easily consume 50% of this chip. We must make strict architectural choices (Area vs. Performance).

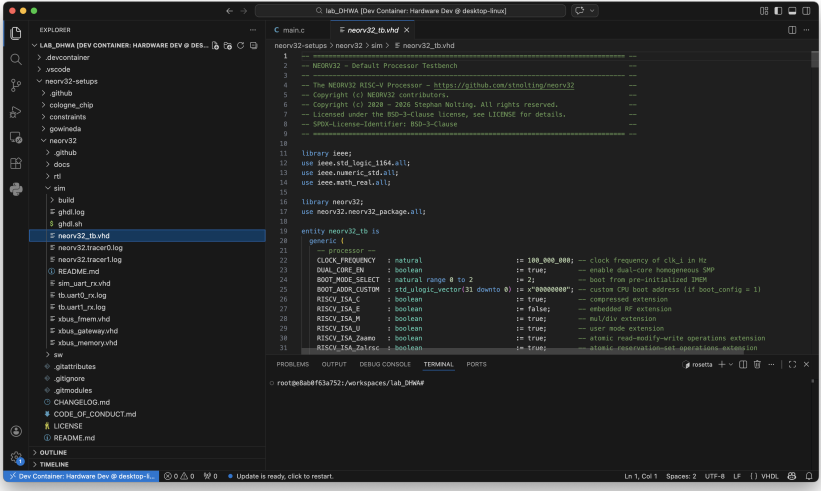
Digital Design & Bring-up Workflow

1. RTL Simulation: Use GHDL to compile the VHDL and simulate the core executing the C code.
2. Waveform Inspection: Use GTKWave/VaporView to debug signals, verify bus transactions, and ensure correct timing before synthesis.
3. Synthesis & PnR: Use Gowin EDA to map the RTL to the physical FPGA resources and route the connections.
4. Bitstream Generation: Create the .fs file.
5. Execution: Upload the bitstream via JTAG.

Note

Today we will execute C tests pre-loaded directly into the bitstream's IMEM (Instruction Memory) for maximum reliability and speed, bypassing the two-stage bootloader.

Development Environment



Hands-on Task: Exploring the Design Space

- Synthesize and test three different NEORV32 configurations by modifying the VHDL top-level generics.
  - Config A (Minimal): No Caches, No M (Multiply) extension.
  - Config B (Memory Optimized): Enable ICACHE (Instruction Cache).
  - Config C (Compute Optimized): Enable ICACHE + M Extension.
- Compile and run two provided C programs for each configuration:
  1. Matrix Multiplication: A compute-heavy workload.
  2. Large Array Sorting: A memory-heavy workload.

Note

Benchmarks are available on WeBeep

Final Goal: Area vs. Performance Trade-off

- For each of the 3 configurations, extract the following metrics:
  - Area: Total LUTs, Flip-Flops, and BRAM used (from the Gowin EDA synthesis report).
  - Performance: Total clock cycles required to complete Benchmark 1 and Benchmark 2 (measured via the NEORV32 internal mtime system timer).

Question

Does the hardware cost (Area overhead) justify the performance gain for both benchmarks? Which configuration is the most "efficient" for each specific task?